

SLAC²: SIMPLES LINGUAGEM ACADEMICA PARA CONSTRUIR COMPILADORES

Manual da linguagem

Leandro Carlos Fernandes

2026

Versão 1.0



Sumário

A linguagem SLAC ²	3
Tipos de dados suportados	3
Declaração de variáveis (simples e vetores)	4
Delimitadores	5
Comentários	5
Palavras reservadas	Erro! Indicador não definido.
Operadores	5
Ordem de avaliação em expressões	6
Blocos de comandos	7
Comandos de entrada e saída (<i>get</i> , <i>echo</i>)	8
○ Comando ECHO:	8
○ Comando GET:	8
Comandos de decisão (<i>case</i> , <i>choose</i>)	9
Comandos de repetição (<i>for</i> , <i>while</i> e <i>repeat until</i>)	10
○ Comando FOR	10
○ Comando WHILE	11
○ Comando REPEAT ... UNTIL	12
Estruturação e seções de um programa em SLAC ²	12
Sub-rotinas (procedimentos e funções)	13
Exemplos de programas em SLAC ²	14
Apêndice A: EBNF da Linguagem SLAC ²	18

SLAC² – *Simple Linguagem Acadêmica para Construir Compiladores*

Trata-se de uma linguagem de programação especificada a partir de objetivos acadêmicos e didáticos, que congrega os principais elementos e estruturas presentes em linguagens comerciais, bem como, endereça os aspectos de interesse relacionados ao controle e execução por arquiteturas de máquinas virtuais e reais.

A linguagem SLAC² tem por objetivo servir de base no aprendizado da construção de compiladores, projetada para ser fácil de analisar e de traduzir para arquiteturas baseadas em pilha (P-máquinas).

Assim, podemos descrever a SLAC² como uma linguagem imperativa com características similares a linguagens tradicionais, como Pascal e C, apresentando tipagem, sintaxe e comandos estruturalmente próximos ao de linguagens comerciais comumente conhecidas por alunos de computação.

Nas subseções que se seguem conheceremos mais sobre a linguagem e trataremos de seus vários elementos constituintes separadamente.

Tipos de dados suportados

Com o objetivo de endereçar os tipos de dados básicos suportados até pelas máquinas mais simples, a SLAC² estabelece apenas de três tipos primitivos para o armazenamento de dados, sendo eles: `int`, `logic` e `chr`, destinados respectivamente para valores inteiros, lógicos e caracteres simples.

Para trabalhar com sequências de caracteres não há um tipo específico, como por exemplo um tipo `string`. No entanto, a linguagem oferece suporte a esse tipo de dados, uma vez que na linguagem as strings são entendidas como vetores de caracteres e que podem armazenar até 255 caracteres.

Em SLAC² uma string é dada por um bloco contíguo de memória em que cada posição armazena o código ASCII dos caracteres que a constitui. O comprimento da sequência é armazenado internamente, sem que haja a necessidade da definição de um caractere terminador (como ocorre em C, por exemplo).

Declaração de variáveis (simples e vetores)

A linguagem SLAC² determina que as variáveis sejam declaradas antes do uso e tenham tipo específico, compatível com a informação que armazenará. São suportadas declarações de variáveis simples ou arranjos unidimensionais homogêneos (vetores).

Sintaxe:

```
<identificador> is <tipo>;
<identificador>, <identificador>, ..., <identificador> is <tipo>;
```

Considerando que a sintaxe determina que a especificação do tipo da variável ocorre no final da instrução, a usual prática de inicializar variáveis no momento da declaração não é um recurso suportado pela linguagem SLAC².

A declaração de vetores segue as mesmas definições, tendo sua sintaxe dada na seguinte forma:

```
<identificador> is <tipo>[<tamanho>];
```

Observe que na SLAC² o índice dos vetores é dado por um número inteiro, cujo valor varia de 1 até o tamanho estabelecido no momento em que o vetor é definido (declaração).

Nota: Na memória, a posição de índice 0 é utilizada guardar o tamanho lógico do vetor, i.e., o comprimento dos dados armazenados. Essa área é reservada para operações de controle e não pertence a região de armazenamento o vetor, de modo que não poderá ser referenciada pelo código do usuário (não se encontra disponível e nem visível ao programador). Trata-se de uma especificação da linguagem que permite a arquitetura da máquina alvo suportar a implementação de arranjos e strings.

Além de conhecer a sintaxe de declaração, é importante compreender onde essas declarações podem ocorrer dentro de um programa SLAC², uma vez que isso determina o escopo e a visibilidade das variáveis. A linguagem organiza as declarações em duas seções distintas: `globvars` e `locvars`.

Variáveis declaradas na seção `globvars`, localizada logo após o cabeçalho do programa, possuem (como o nome sugere) um escopo global, isto é, podem ser acessadas e manipuladas em qualquer parte do programa, incluindo dentro de procedimentos e funções. Essas variáveis existem durante

toda a execução do programa e são frequentemente utilizadas para armazenar informações de uso compartilhado entre diferentes sub-rotinas.

Por sua vez, a seção `locvars` aparece exclusivamente no interior de sub-rotinas – sejam procedimentos ou funções – e serve para definir variáveis cujo uso é restrito àquele bloco específico. Variáveis locais têm escopo limitado à sub-rotina em que foram declaradas e deixam de existir quando sua execução é encerrada.

Essa separação clara entre variáveis globais e locais contribui para a organização do código, reduz a probabilidade de conflitos de nomes e reforça boas práticas de encapsulamento.

Ambas as seções são opcionais e independentes, permitindo ao programador estruturar o armazenamento de dados de acordo com as necessidades lógicas de cada programa.

Delimitadores

Alguns caracteres têm função específica dentro de uma linguagem e determinam a forma como são interpretados pelo compilador. Em SLAC² os caracteres abaixo são utilizados para marcar o início e o fim os seguintes elementos de programação:

"..." Strings
'...' Caracteres

Comentários

Para suportar tanto textos explicativos curtos quanto seções de documentais mais longas é comum que as linguagens de programação disponham de maneiras diferentes para a documentação de código. Como ocorre em outras linguagens, a SLAC² define duas formas básicas para a documentação de código: os comentários de linha única ou comentários de múltiplas linhas.

/# ... #/ Bloco (múltiplas linhas)
... Linha (até o fim da linha)

Operadores

Todas as operações suportadas pela linguagem, quer sejam matemáticas (lógicas e aritméticas) ou de armazenamento (atribuição), são representadas

por operadores específicos. Em geral, as linguagens de programação definem um conjunto de símbolos - formados por um ou mais caracteres - que determinam a operação e a quantidade de operandos envolvidos nela.

A seguir encontraremos a listagem, agrupados por categoria/natureza, dos operandos suportados pela linguagem SLAC²:

Categoria	Operador	Operação
Atribuição	<<	Armazena valor em variáveis
Aritméticos	+	Adição
	-	Subtração
	*	Multiplicação
	//	Divisão de inteiros
Relacionais	=	Igualdade
	~=	Diferença
	>	Maior
	<	Menor
	>=	Maior ou igual
	<=	Menor ou igual
Lógicos	&	Conjunção
		Disjunção
	~	Negação

Ordem de avaliação em expressões

Ao avaliar expressões, a linguagem SLAC² segue uma hierarquia rigorosa de precedência entre os operadores e que estabelece quais operações devem ser processadas primeiro. Isso garante a consistência na avaliação sem que haja uso excessivo de parênteses, enquanto permite que, desejando tornar o código mais inteligível, o uso explícito de parênteses seja feito.

A tabela a seguir resume a precedência de todos os operadores suportados pela linguagem, ordenados do mais prioritário para o menos prioritário. Cabe ainda destacar que todos os operadores binários são associativos à esquerda, enquanto operadores unários (tais como: ~), são associativos à direita.

Categoria	Operador(es)	Descrição / Observação
Unários	~, -	Negação lógica; negativo unário (associativos a direita)
Aritméticos	*, //	Multiplicação e divisão

	+, -	Adição e subtração binária
Relacionais	<, <=, >, >=	Comparações
	=, ~=	Igualdade e diferença
Lógicos	&	Conjunção lógica
		Disjunção lógica (menor precedência entre os binários)

Isto significa que a expressão $a + b > c \ \& \ d = e \ | \ \sim f$ seria avaliada na linguagem SLAC² na seguinte ordem:

1. $a + b$ (adição)
2. $(\text{resultado1}) > c$ (maior)
3. $d = e$ (igualdade)
4. $(\text{resultado2}) \ \& \ (\text{resultado3})$ (conjunção lógica)
5. $\sim f$ (negação)
6. $(\text{resultado4}) \ | \ (\text{resultado5})$ (disjunção lógica)

Essa ordem produz uma leitura determinística da expressão e evita ambiguidades. Finalmente, vale reforçar que parênteses podem ser usados para alterar explicitamente a ordem natural de avaliação, tornando a intenção do programador imediatamente evidente.

Blocos de comandos

Na linguagem SLAC² define blocos de comandos que estruturalmente agrupam várias linhas de comandos e que se comporta como um elemento único. Para construí-lo são usadas as palavras reservadas `start` e `end`, que identificam respectivamente o início e o fim do conjunto de comandos, dados da seguinte forma:

```
start
  <comandos que compõem o bloco>
end
```

Internamente, um bloco de comandos pode conter: nenhum, um ou vários comandos, ou até mesmo, outros blocos de comandos. Vale ressaltar que um bloco de comando vazio, isto é, aquele que não contém nenhum comando, é um trecho de código estruturalmente correto e funcionalmente sem efeito.

Comandos de entrada e saída (*get*, *echo*)

A linguagem SLAC² apresenta um comando para realizar as operações de saída de dados em vídeo - *echo* - e outro para a entrada de dados - *get* -, que varre as informações a partir do teclado. Ambos são descritos com mais detalhes a seguir.

○ *Comando ECHO:*

O comando *echo* é capaz de escrever diferentes tipos de informação na tela, podem ser: literais, caracteres, strings, conteúdo de variáveis, resultados de expressões ou valores retornado de chamadas a sub-rotinas.

Sua utilização é dada no formato:

```
echo( <elemento> ) ;
```

... ou ainda, por:

```
echo( <elemento>, <elemento>, ... , <elemento> ) ;
```

Observe que a forma estrutural do comando suporta a escrita de múltiplos valores simultaneamente, ou seja, é possível escrever vários elementos na tela de uma só vez simplesmente separando-os por vírgulas.

○ *Comando GET:*

Este comando recebe um dado a partir do teclado e o armazena na variável informada como parâmetro. A sintaxe do comando *get* obedece a forma:

```
get( <variável> ) ;
```

As informações do teclado são lidas e transferidas para endereços na memória, portanto, é esperado que apenas variáveis sejam informadas como argumentos nas chamadas de *get*.

Diferentemente do comando de escrita, que suporta múltiplos argumentos, sempre que houver necessidade de ler mais de um dado do teclado, o programador deverá usar um comando *get* para cada leitura.

Comandos de decisão (*case*, *choose*)

Os comandos de desvio controlam o fluxo de execução, direcionando-o para algum outro trecho do código. O destino do desvio é determinado a partir de algum elemento de decisão, que em geral pode ser uma expressão lógica ou o valor de uma variável.

A linguagem SLAC² dispõe de dois comandos condicionais, um simples (*case*) e um composto (*case...otherwise*); além de um comando de seleção (*choose*).

Os comandos condicionais direcionam o fluxo de execução de acordo com o resultado da avaliação de uma expressão lógica. Em SLAC² as estruturas condicionais são construídas com o comando *case*, cuja sintaxe é:

```
case ( <expressão> )
    <comandos> ;
end ;
```

... ou ainda, na forma de um condicional composto, por:

```
case ( <expressão> )
    <comandos>
otherwise
    < comandos> ;
end ;
```

Nota: *O emprego dos parênteses na expressão é obrigatório, fazendo parte da estrutura sintática do comando case.*

O seu funcionamento é simples e idêntico ao que se observa nas demais linguagens de programação. Sempre que a expressão é avaliada como verdadeira o fluxo de execução é direcionado para o comando ou bloco de comandos associado a seção *case*. Caso resulte em falso, o fluxo é direcionado para o primeiro comando subsequente a estrutura do condicional (nos condicionais simples); ou para o comando ou bloco de comandos associado a seção *otherwise* (no caso de condicionais compostos).

Embora seja possível combinar condicionais em arranjos encadeados ou aninhados, dispor de um comando que chaveie o fluxo de execução entre múltiplos destinos possibilita a escrita de códigos mais curtos, legíveis e de fácil compreensão.

O comando *choose* da linguagem SLAC² tem como propósito oferecer um mecanismo estruturado de desvios múltiplos, permitindo ao programador

selecionar diferentes caminhos de execução com base no valor de uma variável ou expressão numérica.

Similar em função aos comandos *switch* de linguagens como C e Java, o comando *choose* tem estruturação mais moderna, com uma sintaxe mais expressiva e flexível, suportando nativamente o uso de listas de valores e intervalos.

```
choose ( <expressão> )
  match <valor1>          -> <comando> ;
  match <valor2> , <valor3> -> <comando> ;
  ...
  others -> <comando> ;
end ;
```

A construção inicia sempre com *choose(expr)*, onde *expr* é avaliada uma única vez; em seguida, cada cláusula *match* define um conjunto de valores possíveis e exatos (ex: 1 ou 2, 3). Quando a expressão se iguala a qualquer uma das alternativas indicadas em um *match*, o comando associado a essa cláusula é executado imediatamente.

O comando pode incluir uma cláusula *others* (opcional), responsável por capturar todos os valores não cobertos pelos itens anteriores e que funciona como um “caminho padrão”, garantindo que o programa tenha um comportamento definido mesmo quando nenhuma alternativa se encaixar.

Comandos de repetição (*for*, *while* e *repeat until*)

Para realizar repetições a linguagem SLAC² dispõe de três estruturas de controle: o comando *for*, empregado quando a quantidade de iterações é pré-determinada; e os comandos *while* e *repeat..until*, para situações em que condicionais governam e determinam quando o término das iterações deve ocorrer.

○ Comando *FOR*

O *for* realiza uma quantidade pré-estabelecida de repetições do comando ou bloco a ele associado. A estrutura do comando tem a seguinte forma:

```
for <varCtrl> from <valorIni> to <valorFim> do
  <comandos> ;
end ;
```

O funcionamento é regido por uma variável de controle, previamente declarada, e que é inicializada com o valor descrito na primeira seção do comando (após o `from`). A cada iteração, a variável de controle é incrementada até que o valor limite (dado pelo número expresso entre `to` e `do`) seja extrapolado.

Por padrão, o incremento é feito de maneira progressiva e na razão de uma unidade (+1). No entanto é possível modificar o passo, estabelecendo outro fator de referência através do elemento `by`. Isso permite variar o fator de incremento com passos que sejam múltiplos de dois, três ou mais; ou ainda, definir uma contagem de forma decrescente empregando algum valor negativo como passo.

Assim, a sintaxe com todos os elementos suportados tem a forma:

```
for <varCtrl> from <valorIni> to <valorFim> by <fator> do
    <comandos> ;
end ;
```

Por exemplo, para construir o laço com uma variável contadora `i` que varie de 10 até 1 passando por todos os valores no intervalo, em SLAC² poderíamos escrever:

```
for i from 10 to 1 by -1 do
```

○ Comando *WHILE*

Com uma estrutura mais simples que o anterior, o `while` é um comando de repetição condicional. Isto é, o processo iterativo é governado por uma expressão condicional e não por um contador.

O comando tem a seguinte estrutura sintática:

```
while ( <expressão> ) do
    <comandos> ;
end ;
```

Essa estrutura de repetição avalia a expressão condicional antes de executar o bloco a ela associado. Assim, a repetição ocorre somente se a expressão dentro dos parênteses resultar em verdadeiro.

Note que se acaso a condição for inicialmente falsa, o comando interno não será executado nenhuma vez, caracterizando o comportamento típico de um laço de pré-teste.

Ao final de cada iteração a expressão é novamente avaliada, determinando se uma nova repetição deve ocorrer ou se o fluxo seguirá para o próximo comando subsequente.

○ *Comando REPEAT ... UNTIL*

Sendo também um comando condicional, o `repeat...until` diferencia-se do `while` em dois aspectos: quanto a sintaxe e quanto ao critério de parada. Aqui a repetição continua enquanto a expressão for falsa; encerrando quando for verdadeira.

Sua sintaxe fica definida da seguinte maneira:

```
repeat
    < comandos> ;
until ( <expressão> ) ;
```

Assim, por empregar duas palavras em sua estrutura sintática, torna-se dispensável a utilização das palavras `start` e `end` para demarcar um bloco de comandos.

Por avaliar a condição apenas ao término do bloco, os comandos localizados entre as palavras `repeat` e `until` terão sido executados uma vez. Neste ponto, o comando decidirá por realizar uma iteração se a condição avaliada ainda não tiver sido satisfeita, ou seja, resultar em falso.

Estruturação e seções de um programa em SLAC²

Os programas na linguagem SLAC² são iniciados pelo procedimento `main()`, que contém o código principal do programa. A seguir encontramos um exemplo com a estrutura mínima e que é comum a todos os programas:

```
proc main()
start
    <comandos> ;
end
```

Em muitas situações os programas precisam manipular dados, quer sejam resultados parciais ou de ler informações do teclado, necessitando assim de

variáveis para armazená-los temporariamente. Como dito anteriormente, as variáveis podem ter escopo global ou local, dependendo do local em que são definidas.

A existência das seções `globvars` e `locvars`, bem como a de quaisquer sub-rotinas além da `main`, é facultativa. Assim, podemos descrever a estrutura de um programa genérico como um modelo mais completo, evidenciando o local de cada seção, como se segue:

```
globvars
    #declarações de variáveis globais

#definição de outras sub-rotinas

proc main()
locvars
    #variáveis locais
start
    #bloco de comandos principal>
end
```

Sub-rotinas (procedimentos e funções)

Como é sabido a linguagem SLAC² suporta a criação de sub-rotinas, sendo definidas em uma região específica do código fonte: entre as variáveis globais e o procedimento principal (`main`).

Habitualmente as sub-rotinas são conhecidas como procedimentos ou funções, diferenciam-se essencialmente como aquelas que não retornam valores (`proc`) daquelas que devolvem algum valor ao ponto de chamada (`func`). Assim há uma ligeira diferença na assinatura de cada uma delas:

```
proc <nomeProcedimento> ( <lstParams> )

func <nomeFunção> ( <lstParams> ): <tipoRetorno>
```

Em SLAC² uma função pode retornar apenas uma informação, compatível algum dos tipos básicos da linguagem. Se a sub-rotina não retorna qualquer valor ao ponto de chamada, deverá ser construída como um procedimento e não uma função.

Quanto aos nomes, o programador pode utilizar qualquer identificador que seja válido na linguagem. Contudo, é sugerida a escolha de um termo condizente com a tarefa realizada pela sub-rotina.

Frequentemente sub-rotinas precisam considerar dados externos para a execução de suas tarefas. Embora a linguagem SLAC² suporte a definição de variáveis globais, o mais comum é definir parâmetros cujos valores serão associados no momento de chamada. A lista de parâmetros de uma sub-rotina é estabelecida em seu cabeçalho, no momento de sua definição. Assim, toda sub-rotina que não necessitar de parâmetros deverá definir uma lista vazia em seu cabeçalho, enquanto as demais devem seguir os seguintes critérios:

- ❖ Cada parâmetro deve ter um tipo associado e ser separado dos demais por uma vírgula. Caso tenhamos mais do que um parâmetro do mesmo tipo, cada um deles deve especificá-lo novamente.
- ❖ Os parâmetros podem ser passados por valor ou por referência.
- ❖ As sub-rotinas podem receber vetores como parâmetros e que serão sempre passados por referência, evitando a necessidade de cópia de dados entre diferentes regiões de memória.
- ❖ O valor dos parâmetros (argumentos) deve ser informado no momento da chamada, respeitando-se a ordem e compatibilidade de tipos que foi especificada no cabeçalho da função, quando da sua definição.
- ❖ Por apresentarem funcionalidade similar a das variáveis locais, os parâmetros estão sujeitos as mesmas regras de escopo.

Nota: A passagem de parâmetros por referência é explicitada pelo emprego da palavra reservada *ref* precedendo o parâmetro em questão, quando da definição da subrotina.

Tal como ocorre no procedimento principal, uma sub-rotina pode ter uma seção para a declaração de variáveis locais. Podemos dizer então que a sintaxe de uma função teria a forma:

```
func <nome> ( <lstParams> ): <tipoRet>
locvars
    #declaração de variáveis locais
start
    #comandos da sub-rotina
    return #valorRetornado ;
end
```

Exemplos de programas em SLAC²

O programa mais simples e mais comum quando da introdução de uma nova linguagem é o “Olá Mundo!”. Assim, nosso primeiro exemplo será a sua codificação em SLAC².

```

proc main()
start
  echo("Olá mundo!");
end

```

Programa 1: Olá Mundo

Mesmo os programas mais simples, realizam algum tipo de entrada e saída de dados, portanto o exemplo que se segue declara uma variável do tipo int para armazenar um valor inteiro digitado pelo usuário e escreve o valor digitado na tela.

```

proc main()
locvars
  x is int;
start
  echo("Entre com um número inteiro:");
  get(x);
  echo("O número que você digitou é:",x);
end

```

Programa 2: Lendo um valor e escrevendo-o na tela.

Neste exemplo, temos um programa que define e invoca uma sub-rotina chamada soma, responsável por realizar a adição dos dois valores inteiros que lhe sejam passados como parâmetro, escrevendo o resultado de seu processamento no vídeo.

```

#função que soma dois valores
func SOMA(A: int, B: int): int
start
  return A+B;
end

#programa principal
proc main()
start
  echo( SOMA(1,2) );
end

```

Programa 3: Exemplo de programa usando funções.

O problema de verificar se, dadas quaisquer três medidas inteiras, elas formam um triângulo e de qual tipo ele seria, é um exemplo interessante para demonstrar a questão do aninhamento de comandos case's, bem como, a utilização dos operadores lógicos nas expressões. Assim, temos:

```

proc main()
locvars
  a,b,c is int;

```

```

start
  echo("Entre com três valores inteiros:");
  get(a); get(b); get(c);
  case (a+b < c | a+c < b | b+c < a)
    echo("Essas medidas não formam um triângulo.");
  otherwise
    case (a = b & b = c)
      echo("As medidas formam um triângulo equilátero");
    otherwise
      case (a ~= b & b ~= c)
        echo("As medidas formam um triângulo escaleno");
      otherwise
        echo("As medidas formam um triângulo isósceles");
      end;
    end;
  end;
end

```

Programa 4: Utilizando condicionais encadeados.

Um bom exemplo da utilização do comando while é o processo de validação de dados de entrada. Suponha que queremos ler um número inteiro digitado pelo usuário e só prosseguir quando esse número estiver no intervalo [1, 100]. Poderíamos implementar a seguinte solução usando linguagem SLAC²:

```

proc main()
locvars
  x is int;
start
  echo("Digite um inteiro entre 1 e 100:");
  get(x);
  while (x < 1 | x > 100) do
    echo("Valor invalido. Digite novamente (1..100):");
    get(x);
  end;
  echo("Valor aceito:", x);
end

```

Programa 5: Validando o intervalo de entrada com o comando WHILE

Suponha um problema onde, dada uma progressão aritmética (PA) com primeiro termo A_1 e razão R , queiramos calcular o somatório dos n primeiros termos.

Talvez uma forma direta e didática, seria calcula cada termo (o i -ésimo termo é $A_1 + (i - 1) * R$) e somarmos os elementos de $i = 1$ até n . Essa estratégia ilustra bem o emprego do for, que nos permite definir o laço com limites explícitos e, se desejado, controlamos o passo.

```

proc main()
locvars

```

```

A1, R, n, i, soma, termo is int;
start
  echo("Primeiro termo (A1):"); get(A1);
  echo("Razão (R):");           get(R);
  echo("Qtde de termos (n):");  get(n);
  soma << 0;
  for i from 1 to n do
    termo << A1 + (i - 1) * R;
    soma << soma + termo;
  end;
  echo("Somatório dos ", n, " primeiros termos da PA: ", soma);
end

```

Programa 6: Determinando a soma dos elementos de uma PA de forma iterativa.

Imagine um programa que determine a prioridade de chamados recebidos em que, dado um código de chamado digitado pelo usuário, o programa classifique sua prioridade segundo uma tabela e reporte a ação adequada. Essa é uma situação interessante para exemplificar o uso comando choose e, se considerarmos que há casos pontuais, listas de valores e intervalos para os códigos, podemos ilustrar as diferentes formas da cláusula match.

```

proc main()
locvars
  cod is int;
start
  echo("Digite o código do chamado:");
  get(cod);
  choose (cod)
    match 0          -> echo("Teste: nenhuma ação necessária.");
    match 9999       -> echo("Encerrar sistema de triagem.");
    match 1, 2, 3    -> echo("URGENTE: acionar equipe de resgate.");
    match 10,11,12   -> echo("Prior. Alta: encaminhar ao n2.");
    match 200, 201   -> echo("Prior. Média: direcionar ao n1.");
    match 500        -> echo("Prior. Baixa: colocar na fila.");
    others           -> echo("Código inválido ou desconhecido.");
  end;
end

```

Programa 7: Usando seletores para classificação de prioridade a partir de múltiplos valores de interesse.

Apêndice A: EBNF da Linguagem SLAC²

Sabendo que os tokens iniciados com “s” representam categorias lexicais, não os lexemas literais da linguagem, podemos definir formalmente a estrutura da linguagem SLAC² por meio da seguinte gramática:

```
G = (N, Σ, P, <prg>)
P:
<prg>    ::= [<gvars>] {<subs>} <princ>

<gvars>  ::= "sGLOBVARS" <decls>{<decls>}
<lvars>  ::= "sLOCVARS" <decls>{<decls>}

<decls>  ::= <id> {", " <id>} "sKIND" <type> "; "
<id>     ::= "sIDENTIF"
<type>   ::= ("sINT" | "sLOGIC" | "sCHR") ["[" "sCTEINT" "]" ""]

<subs>   ::= (<func> | <proc>)

<func>   ::= "sFUNC" <id> "(" [<param> "]" "]" ":" <type> [<lvars>] <bco>
<proc>   ::= "sPROC" <id> "(" [<param> "]" "]" [<lvars>] <bco>

<param>  ::= ["sREF"] <id> ":" <type> {", " ["sREF"] <id> ":" <type>}

<bco>    ::= "sSTART" {<cmd> "; "} "sEND"

<cmd>    := <echo> | <get> | <case> | <chse> | <for> | <whle> |
           <rept> | <call> | <ret> | <atr>

<vetr>   ::= <id> "[" ("sCTEINT" | <id>) "]"

<echo>   ::= "sECHO" "(" (<elem> {", " <elem>} ")"
<get>    ::= "sGET" "(" (<id> | <vetr>) ")"

<case>   ::= "sCASE" "(" (<expr> ) <cmd> ["sOTHERWISE" <cmd>] "sEND"

<chse>   ::= "sCHOOSE" "(" (<expr> ")" <mlst> "sEND"
<mlst>   ::= <mtch> {<mtch>} [<other>]
<mtch>   ::= "sMATCH" <mvls> "sIMPLIC" <cmd> ";"
<mvls>   ::= "sCTEINT" {", " "sCTEINT"}
<othr>   ::= "sOTHERS" "sIMPLIC" <cmd> ";"

<for>    ::= "sFOR" <id> "sFROM" (<id> | "sCTEINT") "sTO" (<id> |
"sCTEINT") ["sBY" "sCTEINT"] "do" <cmd>

<whle>   ::= "sWHILE" "(" (<expr> ")" "sDO" <cmd>
```

```

<rept> ::= "SREPEAT" {<cmd> ";" } "SUNTIL" "(" <expr> ")"

<call> ::= <id> "(" [expr {"," <expr>}] ")"
<ret>  ::= "SRETURN" <elem>

<atr>  ::= (<id> | <vetr>) "SATRIB" <elem>

<elem> ::= <litl> | <id> | <vetr> | <call> | <expr>

<litl> ::= "SSTRING" | "SCTEINT" | "SCTECHAR"

<expr> ::= <elogc> {"SOR" <elogc>}
<elogc> ::= <erlac> {"SAND" <erlac>}
<erlac>  ::= <earit> {<oprel> <earit>}
<earit>  ::= <earip> {<opari> <earip>}
<earip>  ::= <fact> {<oparp> <fact>}

<fact>  ::= <elem> | "SNEG" <fact> | "SSUBRAT" <fact> | "(" <expr> ")"

<oprel> ::= "SMAIOR" | "SMAIORIGUAL" | "SIGUAL" |
           "SMENOR" | "SMENORIGUAL" | "SDIFERENTE"

<opari> ::= "SSOMA" | "SSUBRAT"

<oparp> ::= "SMULT" | "SDIV"

```